

COB-2025-1161

QUADROTOR DYNAMICS IDENTIFICATION USING NEURAL NETWORKS

Otávio Zordan-Alves
Guilherme de Paula Pinheiro
Yohan Díaz-Méndez

Institute of Mechanical Engineering, Federal University of Itajubá, Brazil
d2024007190@unifei.edu.br, d2022009982@unifei.edu.br, yohan.g8@unifei.edu.br

Peter Funk

School of Innovation, Design and Engineering, Malardalen University Västerås/Stockholm, Sweden
peter.funk@mdu.se

Abstract. *In terms of quadrotor control, constructing an appropriate dynamic model is the first step in order to guarantee good controller performance. Due to the inherently unstable, non-linear behavior and significant coupling of quadrotor dynamics, modeling is a challenging task. Mathematical modeling, such as using the Newton-Euler formulation, is not able to capture unobserved states or highly nonlinear effects, such as aerodynamic forces and torques, rotor interactions, or possible system configuration modifications. The main objective of this work is to estimate the flight dynamic response of a quadrotor using neural networks, based on observed data collected in a series of flights and obtained through experimental tests. The neural network is trained with data from different trajectories and validated with others. The platform to test the efficiency of the neural network in learning the dynamic model is a Crazyflie quadrotor. A combination of LSTM Recurrent Neural Network, and Convolutional Network was proposed, trained and tested, showing promising results. The nonlinear quadrotor model obtained showed high accuracy in predicting translational velocities with a good generalization. It was also made a comparison between real 3D trajectory and the estimated by the network showing a great capability of inferring the quadrotor behavior.*

Keywords: *Quadrotor, Recurrent Neural Networks, Convolutional Neural Networks, System Identification, Long Short-Term Memory*

1. INTRODUCTION

Quadrotor drones have gained notable prominence in recent years due to their relatively low cost, mechanical simplicity and vertical take-off and landing (VTOL) capability, which allows the quadrotors to operate in confined spaces and demonstrate impressive maneuverability, including hovering, stable flight, and precise landing (Hoffmann *et al.*, 2007). This versatility and the development of onboard electronic equipment installed on them have enabled their rapid adoption in both civilian and industrial applications. Their use in research and product development has also exponentially increased in the last decade, allowing quadrotors to become more effective in tasks such as infrastructure inspection, mapping terrain, land surveying, search and rescue operations, monitoring wildlife, and mining areas to delivering goods.

A quadrotor drone is an unstable system and must have a control system for its correct operation and safety. It is also under-actuated, nonlinear and difficult to control on aggressive trajectories due to the nonlinearities itself and the presence of perturbations or unmodeled dynamics. Most of the efforts in this area focuses on designing controllers that are derived from a linearization of the model around hover conditions and are stable only under reasonably small roll and pitch angles (Bansal *et al.*, 2016). This approach required the ingenuity of control law designers, starting by gain scheduling for different flight conditions, while keeping the classical and modern linear control methods unchanged. Although linear models are easier to handle which reduces complexity, a popular strategy is to use nonlinear models and controllers. Nonlinear control techniques evolved to reject disturbances and inaccuracies, making them robust and/or adaptive. In all previous cases, to safely control a UAV, it is critical to accurately model the system dynamics to capture the highly nonlinear effects generated by aerodynamic forces and torques, propellers interactions, vibrations, and other phenomena (Saviolo *et al.*, 2022).

The modeling of quadrotor dynamics is commonly classified into three main approaches being the third a combination of the first two: the first one is the Model-Based, that uses principles of physics to derive the differential equations that describe the system's behavior. These techniques have found great applicability in the last decades thanks to the underlying analytical formulation of system dynamics in terms of differential equations, which typically informs the design of performance-guaranteed control techniques (Gu *et al.*, 2020). However, a notable drawback of this approach is

its low precision in predicting the mathematical model of the system under control (Ioannou and Sun, 1996). The second type is Data-Driven method (also known as black box modeling), it relies purely on measured input-output data to build models using statistical or machine learning techniques, without explicitly modeling the physics. By using learning-based methods, the traditional UAV modeling processes such as wind tunnel experiments, computational fluid dynamics (CFD) simulation, and derivation of dynamic equations can be extensively simplified (Jiang *et al.*, 2022). Data-driven methods achieve suitable modelling of errors, uncertainties and perturbations. It is useful to capture the relevant dynamics directly from operational data, that is the main objective of the present work. According to Beintema *et al.* (2023), advances in deep-learning based identification of dynamic systems have resulted in a new generation of nonlinear identification approaches utilizing state-space neural models.

According to Gu *et al.* (2024), the goal of data-driven approaches is to derive the true behavior of complex systems based on observed data by using empirical datasets that, as mentioned before, include uncertainties and disturbances, that are very difficult to take into account in model-based approaches (whose "inefficiency" is generally corrected by using robust or adaptive controllers). Artificial Neural Networks (ANNs), as universal approximators with powerful learning capability, have emerged and gained a lot of interest for UAV dynamical modeling in recent years. Neural Networks are most appropriately applied to model non-linear behaviors, particularly when a mathematical model is not readily available to describe the system, that is the case of quadrotor dynamics in which ANNs also help to model system dynamics and then apply the learned model to adapt a control law.

Modeling dynamical systems is one of most important engineering applications and prediction models can be utilized in order to construct and estimate accurate nonlinear dependence between inputs and outputs. ANNs can model an underlying process directly from input-output data, in the systems and control field, the art of building mathematical models from observed input-output data is called system identification Ljung (2010). Linear models typically require less data and are structured and well-behaved, on the other hand, complex data structures are required to obtain accurate models of highly nonlinear systems. The use of ANNs stands out due to their remarkable ability to model complex nonlinear phenomena Lundby *et al.* (2023). According to Telli *et al.* (2023), the dynamics of quadrotors are characterized by their pronounced non-linearity, under-actuation, and multi-variable nature. These inherent complexities pose challenges to researchers and engineers who aim to develop effective control strategies and navigation systems. Accurate modeling of quadrotor dynamics is pivotal for ensuring stability, control, and optimal performance and ANNs have demonstrated this capability as showed by Avdeev *et al.* (2021), Sifuentes *et al.* (2024) and Atapattu (2020). ANNs encompassing a variety of types including convolutional neural networks (CNN) and recurrent neural networks (RNN), have showcased exceptional capabilities in discerning complex, nonlinear relationships within data and will be explored in the present work.

In Punjani and Abbeel (2015) and Abbeel *et al.* (2010), ANNs have been used for modelling complex aerodynamics phenomena and vibrations, usually neglected, for helicopter flight control. Shi *et al.* (2019) applied a Deep Neural Network (DNN) to model ground effects and proposes a DNN-based controller that enhances the landing behavior of drones. On the other hand, Gu *et al.* (2024) used a strategy based on combination of model-based and data-driven approaches called, Physics-Informed Machine Learning (PIML), for dynamical modeling of quadrotors. This method can achieve a better generalization capability by incorporating a-priori system knowledge into the learning process. In the same line of knowledge, Saviolo *et al.* (2022) proposed a Physics-Inspired Temporal Convolutional Network (PI-TCN) that combines a temporal convolutional network that encodes time-correlated features from a history of past states and control inputs with a multi-layer perceptron demonstrating accurately results capturing hidden effects of quadrotor dynamics.

When modeling time-series data, recurrent neural networks (RNNs) provide better architectures for sequential data modeling than feedforward classical networks (FFNNs). Despite FFNNs allowing highly complex phenomena to be modeled, they are not designed for learning-time-correlated features. In a classical recurrent neural network (RNN) layer, nodes are replaced by memory cells that receive at time t the current input data in addition to their own output produced at the time $t - 1$. Due to these cells, RNNs can identify temporal dependencies in sequential data (Zhao *et al.*, 2019). RNNs face significant challenges when dealing with long-term dependencies, mainly due to vanishing and exploding gradient problems during backpropagation over time (Gu *et al.*, 2024). These effects impair the network's ability to capture relevant information occurring at distant points in the sequence, compromising performance in tasks that require extended memory. As an alternative, other types of memory cells can be used, including long- and short-term memory (LSTM) (Altena *et al.*, 2023).

Temporal Convolutional Networks (TCNs) have also been explored to tackle this issue due to their ability to process long sequences more stably. TCNs use causal and dilated convolutions to capture long-range temporal dependencies without suffering from the typical gradient issues associated with RNNs. Additionally, their parallelizable architecture contributes to more efficient training, making them a promising approach for modeling time series and complex sequential signals. CNNs have also been reduced the dimensionality of high-dimensional systems (Jiahao *et al.*, 2021) that allows to detect spatial or temporal correlations in local neighborhoods, which makes them particularly suitable for detecting local behavior or instabilities in quadrotor flight data, extracting motion features and capturing short-term temporal dependen-

cies in sequences.

In the present work, a type of RNN, known as LSTM is used to model the nonlinear behavior of a quadrotor drone using data from experiments in which the quadrotor platform executed complex maneuvers. To complement LSTM's ability to capture long-term dependencies, convolutional neural networks (CNNs) are also employed to extract local and frequency-based patterns from the input signals. Our model differs from other similar approaches because leverages the sequential structure of the data to capture nontrivial couplings between inputs and outputs, being the last ones, linear and angular velocities instead accelerations adopted in previous works.

2. QUADROTOR NEURAL NETWORK MODEL

2.1 Quadrotor Description

A 6-DOF quadrotor has twelve states: the position on each axis in the world frame (x, y, z) , the Euler angles (ϕ – roll, θ – pitch, ψ – yaw), the body velocities (u, v, w) and the angular velocities (p, q, r) . It has four inputs, that are related to the thrust force (u_z) and torques on each axis $(u_\phi, u_\theta, u_\psi)$ Rocha *et al.* (2025). In our case, the collected data of control inputs are the *PWM* signals sended by the controller board to the motors. Axis labels and rotational conventions as well as the defined frames of reference of the model (W – inertial/earth and B – body) are shown in Figure 1.

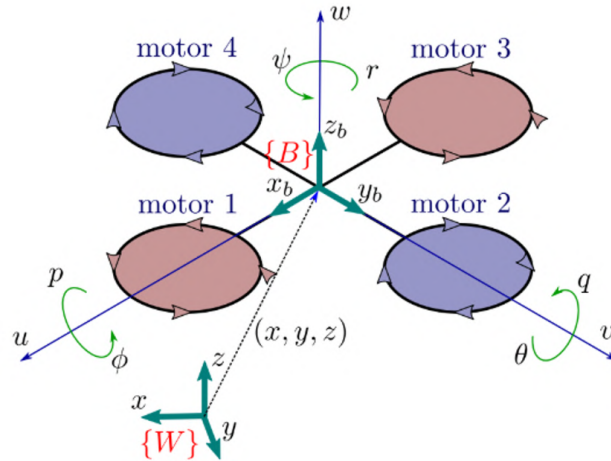


Figure 1: Quadrotor reference frames and states (Rubí *et al.*, 2021)

To execute the several 3D trajectories during the data collection phase, the original cascade Proportional Integral Derivative (PID) control architecture of the quadrotor platform, crazyflie, was adopted (see Fig. 2).

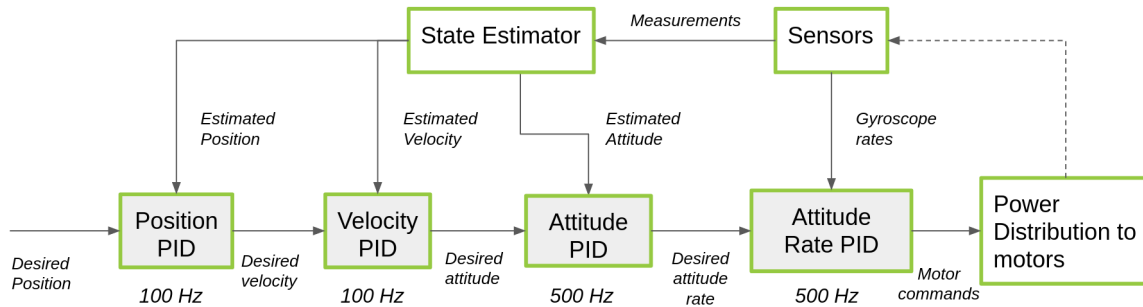


Figure 2: Cascade PID controller (Bitcraze, 2025)

The PID control architecture of the Crazyflie is composed of 4 PID controllers, each of them controls a different aspect of the drone, and helps to obtain the desired motor control parameters: the first one (from right to left of Fig. 2), called attitude rate, computes the motor commands from the rate error between desired and estimated Roll-Pitch-Yaw (RPY) rates. In turn, the desired rotational rates are calculated by the attitude controller (second PID) which uses as inputs, the desired and estimated pitch, roll and yaw rotational positions. The third controller, called velocity PID follows the same procedure, updating the desired attitude by receiving the desired velocity from the position PID (fourth PID), which receives and processes the user predefined XYZ trajectory.

2.2 Neural Network Model

Inspired by Bansal *et al.* (2016), which proposes the use of neural networks for identifying and modeling quadrotor dynamics from input-state data, this work explores a hybrid LSTM-CNN architecture aimed at learning, in a supervised fashion, to predict dynamic system variables, angular and linear velocities, from motor command signals. Unlike approaches that model accelerations, we opt to predict angular velocities $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ and linear velocities $(\dot{x}, \dot{y}, \dot{z})$, as these are directly measurable via the LightHouse system (local positioning system) and the IMU (Inertial Measurement Unit). This choice avoids the noise amplification typically associated with numerical integration of accelerations and ensures compatibility with control strategies required in applications we plan to develop in the future.

Formally, our aim is to learn a mapping of the form stated by Equation 1.

$$\hat{\mathbf{y}} = \mathbf{h}_{\theta}(\mathbf{u}), \quad \mathbf{u} \in \mathbb{R}^{4 \times T}, \quad \hat{\mathbf{y}} \in \mathbb{R}^{6 \times T} \quad (1)$$

where $\mathbf{u}$ denotes the normalized PWM signals of the four motors over T time steps, and $\hat{\mathbf{y}}$ represents the predicted angular and linear velocities, three angular components $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ and three linear ones $(\dot{x}, \dot{y}, \dot{z})$.

Long-Short-Term Memory (LSTM) networks are a type of recurrent neural network (RNN) specifically designed to handle temporal sequences and preserve relevant information over long time intervals. Each LSTM cell maintains a memory vector c_t , which is updated through three gates: the input gate i_t , the forget gate f_t , and the output gate o_t . These gates are defined by sigmoid functions acting on the current input x_t and the previous hidden state h_{t-1} . The memory update is given by the function described by Equation 2.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (2)$$

Here, $\tilde{c}_t$ is the candidate cell state, obtained through a nonlinear transformation of the current input concatenated with the previous hidden state. The forget gate f_t controls how much of the previous memory c_{t-1} is retained, while the input gate i_t governs how much of the new information $\tilde{c}_t$ is incorporated. The operator $\odot$ denotes element-wise multiplication (Hadamard product). The hidden state h_t is then updated based on the new memory c_t , modulated by the output gate o_t (see Eq. 3).

$$h_t = o_t \odot \tanh(c_t) \quad (3)$$

This structure allows the LSTM to dynamically decide, at each time step, what to store, update, or discard from its internal memory, enabling the network to capture long-range temporal dependencies. At each step t , the LSTM processes the input x_t , representing the PWM commands of the four motors at that instant, and updates its internal states c_t and h_t , propagating the information through the input window of length T . The final hidden state h_T encodes the accumulated information over the entire sequence.

To complement LSTM's ability to capture long-term dependencies, convolutional neural networks (CNNs) are employed to extract local and frequency-based patterns from the input signals. Rather than connecting every neuron across adjacent layers, CNNs apply filters (or kernels) that slide over the time sequence, performing localized convolutions. This one-dimensional convolution operation can be expressed as stated by Eq. 4.

$$s(t) = (x * w)(t) = \sum_{i=0}^{k-1} x(t+i) \cdot w(i) \quad (4)$$

where k is the filter size, $x(t)$ is the input sequence at time t , $w(i)$ is the weight vector of the filter, and $s(t)$ is the output of the convolution at time t . The result is typically passed through a nonlinear activation function such as ReLU, defined as $\text{ReLU}(z) = \max(0, z)$, which introduces nonlinearity into the model. Pooling layers may follow, reducing the dimensionality of the representation and making it more robust to local variations and input noise.

After parallel processing by the LSTM and CNN branches, their extracted features are concatenated, forming a combined feature vector that captures both temporal dependencies and local signal patterns. This joint representation is then passed through fully connected dense layers, which serve to refine and merge the learned information, improving the regression of the target dynamic variables.

The data used for training were sampled at 100 Hz, in time windows of $T = 100$ samples, corresponding to 1 second of flight. This window-based approach enables the model to learn accumulated effects and short-to-medium-term dynamics. Unlike methods such as Bansal *et al.* (2016), which rely solely on the current state as input, our model leverages the sequential structure of the data to capture nontrivial couplings between inputs and outputs.

Finally, as stated before, the model's output is a six-dimensional vector representing the target state variables: three body-frame angular velocities $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ and three global-frame linear velocities $(\dot{x}, \dot{y}, \dot{z})$. The described neural network architecture is illustrated at Figure 3.

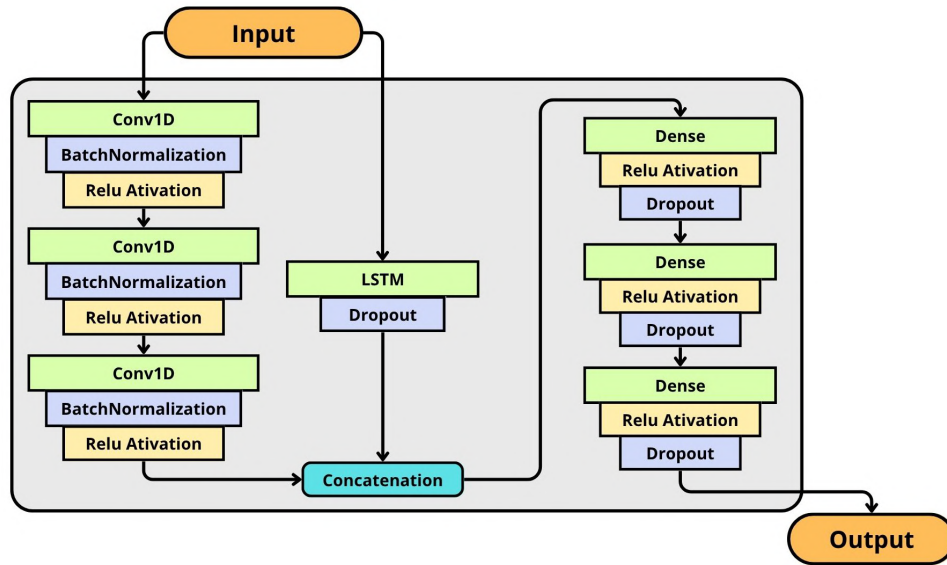


Figure 3: Parallel LSTM-CNN architecture, with LSTM and CNN branches concatenated for velocity prediction.

3. METHODOLOGY

This section outlines the development process of a neural network model designed to predict the angular and linear velocities of the Crazyflie drone from motor commands. The process includes five main stages: data collection, preprocessing, model architecture definition, training, and evaluation.

Data acquisition was conducted using the Crazyflie 2.0 drone, autonomously controlled via the `cflib` Python package, which enables radio communication and the transmission of high-level positioning and control commands. The LightHouse positioning system, calibrated using `cfclient`, was used to record precise measurements of translational velocities (`stateEstimate.vx`, `vy`, `vz`) in the global frame and rotational velocities (`gyro.x`, `gyro.y`, `gyro.z`) in the body frame, both at a rate of 100 Hz. Motor commands (`motor.m1`, `m2`, `m3`, `m4`), represented as PWM signals, were recorded simultaneously, enabling correlation between control inputs and the drone's dynamic responses. A total of 240 flights were stored as CSV files for later preprocessing and model training. Crazyflie platform and data collection process is depicted at Figure 4.

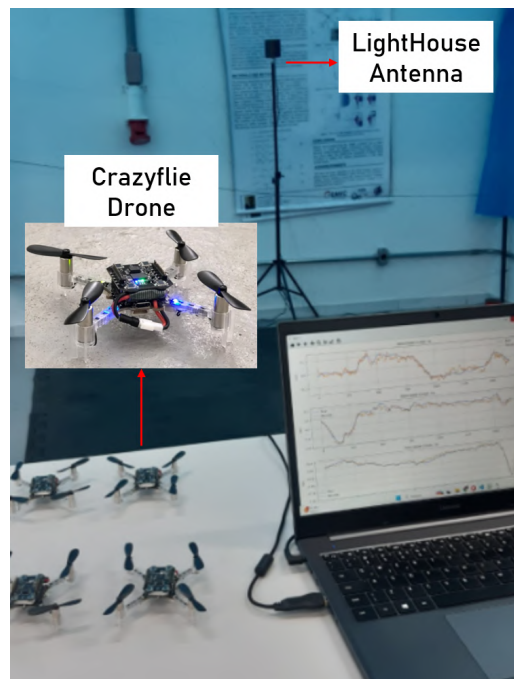


Figure 4: Data Collection, crazyflie 2.0 quadrotor, lighthouse positioning system and computer interface.

Prior to data collection, a detailed mapping of the lab was performed to identify areas with optimal LightHouse system

coverage, which uses two infrared base stations (antennas) for high-precision tracking. A Python script was developed to evaluate marker visibility across different regions, considering the station layout at the corners of the lab. Areas with poor coverage, such as wall-adjacent corners or obstructed regions, were identified and avoided in the flight scripts. The spatial boundaries were defined as a cuboid with coordinates $x \in [0.7, -0.2]$, $y \in [-0.7, 0.15]$, and $z \in [0.3, 0.8]$, ensuring all trajectories remained within a high-precision tracking zone. This step was essential to minimize positioning errors and ensure data quality. Figure 5 illustrates the flight cage used in the experiments.

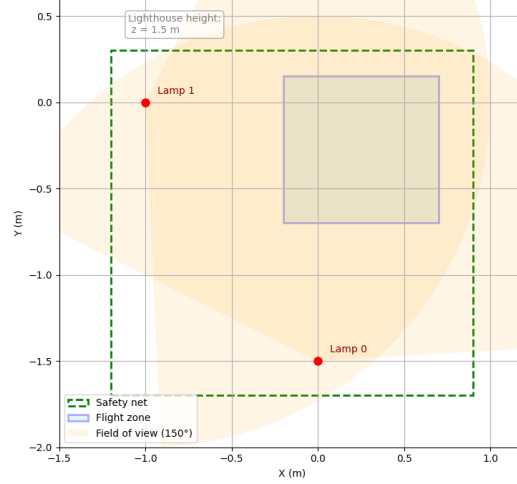


Figure 5: Top-view schematic of the flight cage used in the experiments.

The blue shaded area represents the high-precision flight zone where all trajectories were executed. The green dashed rectangle indicates the physical safety net that surrounds the setup. Red circles indicate the positions of the Lighthouse V2 base stations, each emitting an optical tracking field of view of 150° , shown in orange. The center of the flight zone is aligned with the origin of the coordinate system (0, 0), and both lighthouses are positioned at a height of $z = 1.5$ m.

The flight trajectories were designed based on Saviolo *et al.* (2022), which employs complex maneuvers to capture nonlinear quadrotor dynamics. A total of 12 distinct sinusoidal spatial trajectories were implemented (see Fig. 6), including lemniscates (vertical, horizontal, twisted, and extended), inclined ellipses, deformed ellipses, horizontal helices, and saddle-shaped paths. These trajectories, collected between October 28, 2024, and May 7, 2025, were chosen to encompass a wide range of flight conditions, from slow movements (0.2 m/s) to more aggressive ones (1.5 m/s), thus promoting model generalization. The choice of trajectories such as deformed ellipses and saddle paths reflects the need to evaluate the drone’s ability to track curves with dynamic variations and to handle additional stability challenges.

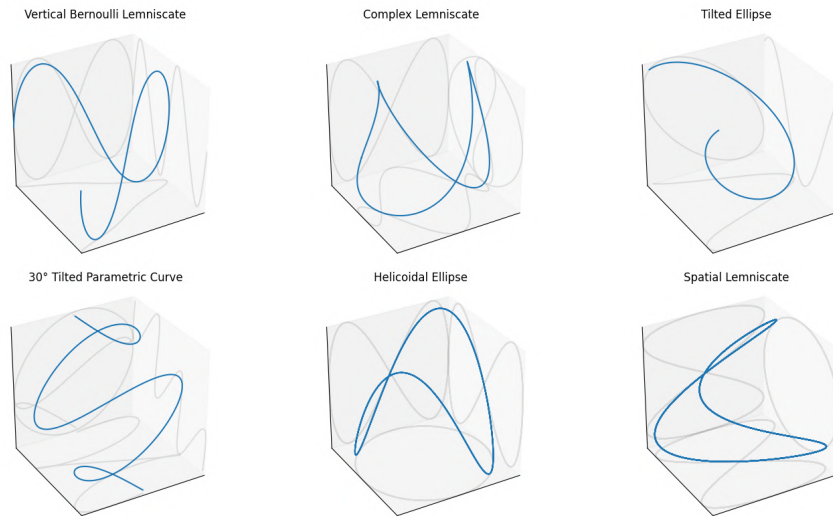


Figure 6: Some parametric 3D trajectories used during data collection. The plots include spatial curves such as vertical lemniscates, helicoidal ellipses, and inclined ellipses, designed to excite different flight dynamics of the quadrotor. Projections onto the XY, XZ, and YZ planes are shown to illustrate motion complexity.

Each trajectory was defined by a set of discrete points in 3D space, generated from custom parametric functions

implemented in Python. For example, the inclined ellipse was created by computing $n = 200$ points with semi-axes $a = 2$ and $b = 1$, inclined at 30° with respect to the XY plane and adjusted to fit within the defined cuboid. For each new flight, random values were added to the parameters used to generate the trajectory, including the number of points, to induce variety and capture more general motion relationships.

Each flight began with the quadrotor in a stable hover condition to ensure consistent initial states across different trajectories. After stabilization, the predefined spatial trajectory was executed. Additionally, to enhance model generalization and prevent bias toward specific orientations, the entire trajectory was randomly rotated in three-dimensional space before each flight. This randomization in orientation ensured a broader excitation of dynamic modes and increased the diversity of flight conditions seen by the model.

The data were logged into CSV files for each flight using the `cflib` logging functionality, configured to capture the relevant variables at 10 ms intervals. Each flight was monitored in real-time through a 3D animation using `matplotlib`, displaying both the planned trajectory and the drone's current position.

Normalization was applied to standardize the scale of the input and output variables, facilitating model training and improving numerical stability. Following Bansal *et al.* (2016), motor commands (`motor.m1` to `m4`) and velocities (`gyro.x`, `y`, `z`, `stateEstimate.vx`, `vy`, `vz`) were globally normalized using standard scaling, adjusting the data to have zero mean and unit variance:

$$z = \frac{x - \mu}{\sigma}, \quad (5)$$

where μ is the mean and σ is the standard deviation computed over the entire dataset. Two `StandardScaler` objects from the `scikit-learn` library were fit separately to the inputs ($u \in \mathbb{R}^4$, motor commands) and outputs ($y \in \mathbb{R}^6$, velocities), with parameters saved for use during inference. This ensured that the differing magnitudes of PWM signals (0 to 65535) and velocities (in rad/s and m/s) were harmonized, promoting efficient training convergence.

To capture temporal dependencies required by the LSTM-CNN architecture, each flight's data were segmented into temporal windows of 100 samples, corresponding to 1 second of flight at 100 Hz. Each input window consists of a sequence of motor commands $u \in \mathbb{R}^{4 \times 100}$, and the corresponding output is the next state $y \in \mathbb{R}^6$, representing the velocities at the time step immediately following the window. This approach, unlike Saviolo *et al.* (2022), which uses sequential outputs, was chosen to predict the next state based on the history of commands, simplifying the prediction task.

Additionally, the input data is first permuted to rearrange the temporal dimension, followed by a single LSTM layer with 128 units and a dropout rate of 30% to mitigate overfitting. In the convolutional branch, three sequential `Conv1D` layers are employed: the first with 128 filters and a kernel size of 8, the second with 256 filters and a kernel size of 5, and the third with 128 filters and a kernel size of 3. Each convolutional layer is followed by batch normalization and a ReLU activation function. A `GlobalAveragePooling1D` layer is then applied to reduce dimensionality. The outputs from both branches are concatenated and passed through two fully connected layers with 64 and 32 neurons, respectively, each with ReLU activation and a dropout of 30%. The final output layer is a linear `Dense` layer with 6 neurons, corresponding to the predicted angular and linear velocities. The model was implemented using the TensorFlow library, and trained using the Adam optimizer (learning rate $\eta = 10^{-3}$) and the mean squared error (MSE) loss function. A validation set comprising 20% of the training data was used for monitoring. The dataset was split into 70% for training and 10% for testing.

4. RESULTS AND DISCUSSION

To evaluate the performance of the proposed LSTM-CNN model, standard regression metrics were used: root mean square error (RMSE), mean absolute error (MAE), and coefficient of determination (R^2) for each output variable. Table 1 summarizes the results obtained on the test set.

Table 1: Performance metrics for each predicted variable.

Variable	RMSE	MAE	R^2
$\dot{\phi}$ (gyro.x)	9.09 rad/s	7.10 rad/s	0.742
$\dot{\theta}$ (gyro.y)	6.58 rad/s	5.31 rad/s	0.458
$\dot{\psi}$ (gyro.z)	3.10 rad/s	2.27 rad/s	0.737
$\dot{x}$ (vx)	0.049 m/s	0.040 m/s	0.963
$\dot{y}$ (vy)	0.055 m/s	0.044 m/s	0.972
$\dot{z}$ (vz)	0.077 m/s	0.060 m/s	0.944

It is observed that the model shows high accuracy in predicting translational velocities, especially along the y axis, with R^2 values above 0.96 for all linear components. This demonstrates the model's ability to generalize well when estimating translational dynamics from motor commands.

In contrast, the angular velocity predictions show moderate performance. While $\dot{\phi}$ and $\dot{\psi}$ achieve R^2 values close to 0.74, $\dot{\theta}$ presents a lower value (0.46), which may be associated with low excitation of the pitch axis in the performed trajectories or with sensor noise.

Figures 7 and 8 illustrate the predictions of angular and linear velocities, respectively, with absolute error bands over time. It is noted that the model captures the general motion trends, although more pronounced oscillations are visible in the angular predictions, especially during more aggressive maneuvers.

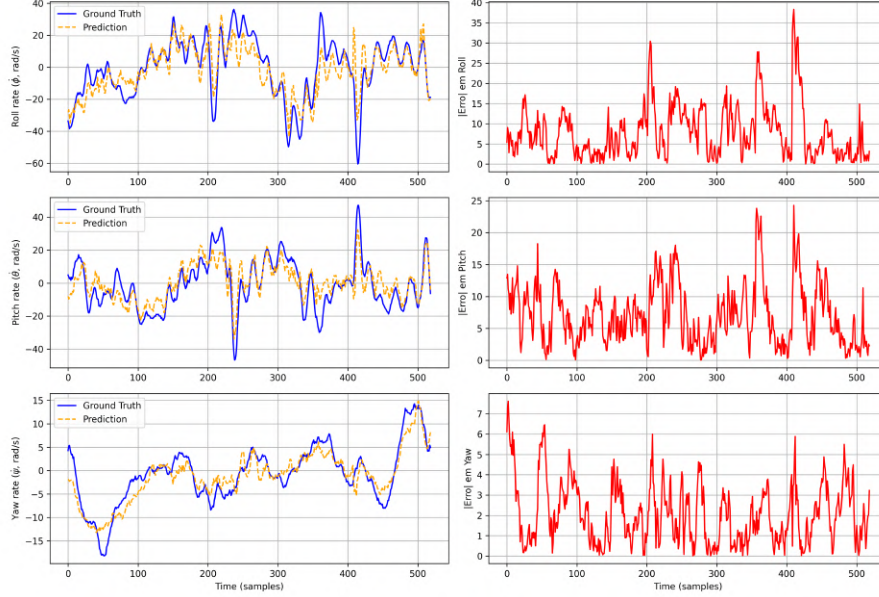


Figure 7: Predictions of angular velocities ($\dot{\phi}$, $\dot{\theta}$, $\dot{\psi}$) with corresponding absolute error bands.

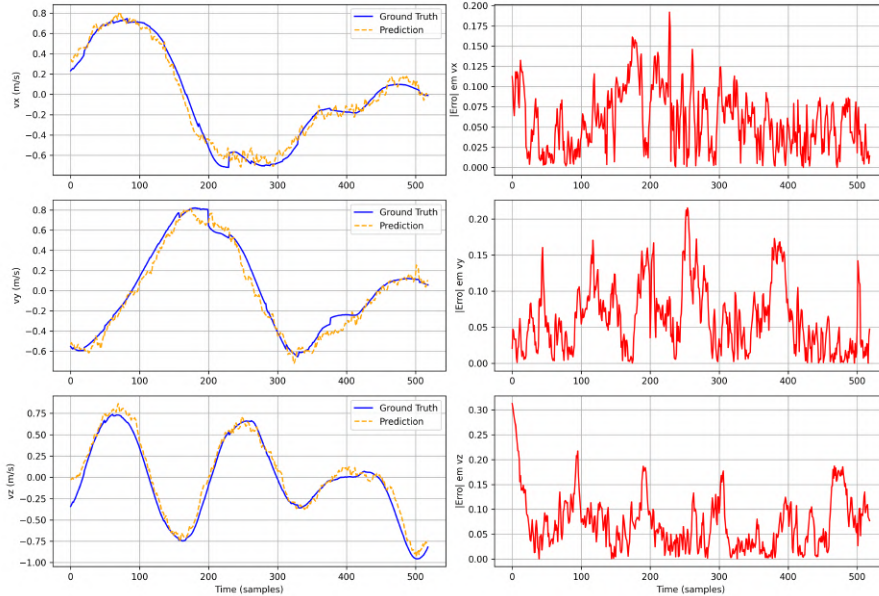
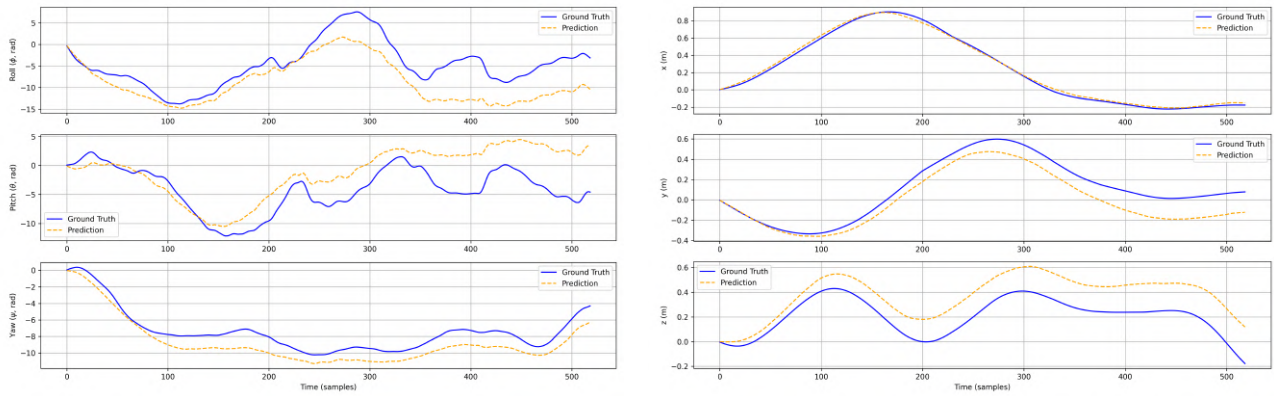


Figure 8: Predictions of linear velocities ($\dot{x}$, $\dot{y}$, $\dot{z}$) with corresponding absolute error bands.

To assess the model's applicability to trajectory tracking tasks, numerical integration of the predicted velocities was performed. Figures 9 shows the reconstructed trajectories from the integration of angular and translational velocities, respectively. Despite the accumulation of drift over time, an expected limitation due to the nature of integration, the trajectories preserve the qualitative structure of the original movements.

Finally, Figure 10 compares the predicted 3D trajectory with the actual trajectory for one of the test cases. The predicted trajectory is represented by a dashed line and the actual trajectory by a solid line for a test flight. The estimated



(a) Angular trajectories reconstructed via integration of predicted velocities.

(b) Translational trajectories reconstructed via integration of predicted velocities.

Figure 9: Quadrotor trajectories reconstructed via integration of predicted velocities.

trajectory remains close to the real one, indicating that the model is capable of faithfully inferring motion from motor commands.

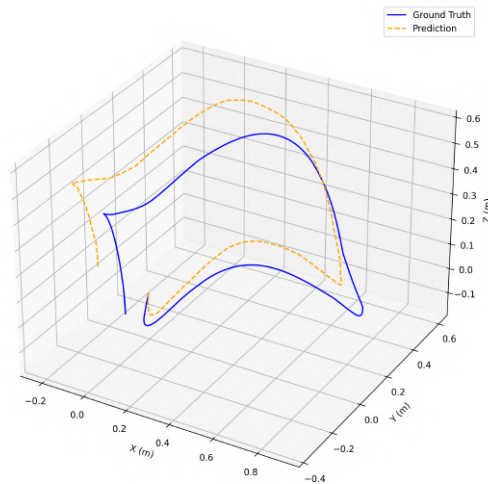


Figure 10: Comparison between the predicted and real 3D trajectory

5. CONCLUSIONS

In this work, CNN architecture was proposed to learn the dynamics of a quadrotor from data by using convolutional networks in conjunction with the LSTM strategy. Several real data, especially, linear and rotational velocities, were collected, normalized, and used to train the proposed deep neural network. The hybrid LSTM-CNN architecture proved to be effective in modeling the flight dynamics of a quadrotor from real-world data. Its accuracy in estimating linear velocities is particularly noteworthy. Improvements in angular velocity predictions may be achieved with greater maneuver diversity, noise filtering techniques, or by introducing auxiliary state variables. One limitation of the proposed approach is the computational time required to train the network; therefore, future works will be focused on the identification and selection of training data that reduces the computational time without generalization and accuracy loss.

6. ACKNOWLEDGEMENTS

This work was carried out at the Aeronautical Systems Laboratory (LSA) and has been supported by the State Funding Agency of Minas Gerais, Brazil (FAPEMIG), Grant Number: APQ-01656-21. This work was also funded by the Programa de Educação Tutorial de Mechanical Engineering (PET/EME) of the Federal University of Itajubá (UNIFEI).

The authors express special thanks to Engineer Eduardo Henrique Teixeira (M.Sc. in Electrical Engineering from the National Institute of Telecommunications – INATEL, Brazil) for his valuable technical contributions and support during the development of this work.

7. REFERENCES

- Abbeel, P., Coates, A. and Ng, A.Y., 2010. “Autonomous helicopter aerobatics through apprenticeship learning”. *The International Journal of Robotics Research*, Vol. 29, No. 13, pp. 1608–1639.
- Altena, A.V., van Beers, J.J. and de Visser, C.C., 2023. “Loss-of-control prediction of a quadcopter using recurrent neural networks”. *Journal of Aerospace Information Systems*, Vol. 20, No. 10, pp. 648–659.
- Atapattu, S.H., 2020. *Estimating unknown dynamics of a quadrotor aerial vehicle artificial neural networks*. Ph.D. thesis, Memorial University of Newfoundland.
- Avdeev, A., Assaleh, K. and Jaradat, M.A., 2021. “Quadrotor attitude dynamics identification based on nonlinear autoregressive neural network with exogenous inputs”. *Applied Artificial Intelligence*, Vol. 35, No. 4, pp. 265–289.
- Bansal, S., Akametalu, A., Jiang, F., Laine, F. and Tomlin, C., 2016. “Learning quadrotor dynamics using neural network for flight control”. *arXiv preprint arXiv:1610.05863*. doi:10.48550/arXiv.1610.05863.
- Beintema, G.I., Schoukens, M. and Tóth, R., 2023. “Deep subspace encoders for nonlinear system identification”. *Automatica*, Vol. 156, p. 111210.
- Bitcraze, 2025. “Controllers — crazyflie firmware documentation (master)”. <https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/sensor-to-control/controllers/>. Accessed: 2025-05-01.
- Gu, W., Primatesta, S. and Rizzo, A., 2024. “Physics-informed neural network for quadrotor dynamical modeling”. *Robotics and Autonomous Systems*, Vol. 171, p. 104569.
- Gu, W., Valavanis, K.P., Rutherford, M.J. and Rizzo, A., 2020. “Uav model-based flight control with artificial neural networks: A survey”. *Journal of Intelligent & Robotic Systems*, Vol. 100, No. 3, pp. 1469–1491.
- Hoffmann, G., Huang, H., Waslander, S. and Tomlin, C., 2007. “Quadrotor helicopter flight dynamics and control: Theory and experiment”. In *AIAA guidance, navigation and control conference and exhibit*. p. 6461.
- Ioannou, P.A. and Sun, J., 1996. *Robust adaptive control*, Vol. 1. PTR Prentice-Hall Upper Saddle River, NJ.
- Jiahao, T.Z., Hsieh, M.A. and Forgoston, E., 2021. “Knowledge-based learning of nonlinear dynamics and chaos”. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, Vol. 31, No. 11.
- Jiang, B., Li, B., Zhou, W., Lo, L.Y., Chen, C.K. and Wen, C.Y., 2022. “Neural network based model predictive control for a quadrotor uav”. *Aerospace*, Vol. 9, No. 8, p. 460.
- Ljung, L., 2010. “Perspectives on system identification”. *Annual Reviews in Control*, Vol. 34, No. 1, pp. 1–12.
- Lundby, E.T.B., Rasheed, A., Halvorsen, I.J., Reinhardt, D., Gros, S. and Gravedahl, J.T., 2023. “Deep active learning for nonlinear system identification”. *arXiv preprint arXiv:2302.12667*.
- Punjani, A. and Abbeel, P., 2015. “Deep learning helicopter dynamics models”. In *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE, pp. 3223–3230.
- Rocha, L., Pinheiro, G., Gomes, G. and Díaz, Y., 2025. “Passive fault tolerant control of quadrotors using nonlinear controller: An experimental approach”. *XX International Symposium on Dynamic Problems of Mechanics*. doi: 10.26678/ABCM.DINAME2025.DIN2025-0101.
- Rubí, B., Morcego, B. and Pérez, R., 2021. “Deep reinforcement learning for quadrotor path following with adaptive velocity”. *Autonomous Robots*, Vol. 45, No. 1, pp. 119–134.
- Saviolo, A., Li, G. and Loianno, G., 2022. “Physics-inspired temporal learning of quadrotor dynamics for accurate model predictive trajectory tracking”. *IEEE Robotics and Automation Letters*, Vol. 7, No. 4, pp. 10256–10263. doi: 10.1109/LRA.2022.3192609.
- Shi, G., Shi, X., O’Connell, M., Yu, R., Azizzadenesheli, K., Anandkumar, A., Yue, Y. and Chung, S.J., 2019. “Neural lander: Stable drone landing control using learned dynamics”. In *2019 international conference on robotics and automation (icra)*. IEEE, pp. 9784–9790.
- Sifuentes, H.E., Rocha, C.A.R. and Ramos, E.M., 2024. “Non-linear dynamic behavior identification of a quadcopter f450 using an artificial neural network-based nars model”. *Ingeniería e Investigación*, Vol. 44, No. 3, p. 6.
- Telli, K., Kraa, O., Himeur, Y., Boumehraz, M., Atalla, S., Mansoor, W. and Ouamane, A., 2023. “Quadrotor experimental dynamic identification with comprehensive narx neural networks”. In *2023 6th International Conference on Signal Processing and Information Security (ICSPIS)*. IEEE, pp. 167–172.
- Zhao, R., Yan, R., Chen, Z., Mao, K., Wang, P. and Gao, R.X., 2019. “Deep learning and its applications to machine health monitoring”. *Mechanical Systems and Signal Processing*, Vol. 115, pp. 213–237.

8. RESPONSIBILITY NOTICE

The authors are solely responsible for the printed material included in this paper.